Got it — you're asking for **"compatible feature examples"** (most likely **backward compatibility**).
I'll explain **clearly + with simple code examples**, mainly in **Java**, since that's your usual context.

---

**1 Backward Compatible Feature (MOST COMMON)**

**New version works with old code**

**✅ Example: Java 8 default methods in interfaces**

Old code **does NOT break**.

```
interface Vehicle {
   void run();

   // Java 8 added feature
   default void fuel() {
      System.out.println("Petrol");
   }
}
```

Old implementation still works:

```
class Bike implements Vehicle {
   public void run() {
      System.out.println("Bike running");
   }
}
```

✔Old classes compile

✔No code change needed

➡ **Backward compatible**

---

**2 Non-Backward Compatible Feature**

**Old code breaks in new version**

**❌ Example: Method signature change**

Before:

```
void calculate(int a)
```

After:

void calculate(int a, int b)

 Old callers break

➡ **Not backward compatible**

---

**3 Backward Compatibility Example (Collections)**

**Java 5 Generics (Type Safety added)**

Old code:

```
List list = new ArrayList();
list.add("Java");
```

New style:

```
List<String> list = new ArrayList<>();
list.add("Java");
```

✔Old raw types still allowed

➡ **Backward compatible**

---

**4 JVM Level Compatibility Example**

**Bytecode Compatibility**

```
// Compiled with Java 8
public class Test {
   public static void main(String[] args) {
      System.out.println("Hello");
   }
}
```

✔ Runs on Java 11 / 17 JVM

➡ **Backward compatible JVM**

✕ Java 17 compiled code may NOT run on Java 8 JVM

---

### 5 API Compatibility Example

**Adding a new method (safe)**

```
class UserService {
   void saveUser() {}
}
```

Later:

```
class UserService {
   void saveUser() {}
   void deleteUser() {}   // new feature
}
```

✔Existing code unaffected

➡ **Compatible change**

---

### 6 Real-Life Example (Simple)

| Change | Compatible? | Reason |
|---|---|---|
| **Adding new button in app** | ✅ Yes | Old users still work |
| **Removing existing button** | ❌ No | Users break |
| **Adding optional field in API** | ✅ Yes | Old clients ignore it |
| **Renaming API field** | ❌ No | Clients fail |

---

### 7 One-Line Definition (Interview-Ready)

**Backward compatibility** means *new software versions can run old code without modification*.

If you want, I can also give:

- □ **Forward compatibility examples**
- □ **Binary vs Source compatibility**
- □ **Java version-wise compatibility table**
- □ **Interview Q&A on compatibility**

Just tell me □